

Google Colab

Experiment Guide

*Running the roberta-base LoRA Rank Sweep
on Google Colab — Step-by-Step*

Experiment : LoRA rank sweep $r = [1, 2, 4, 8, 16, 32, 64]$ + full fine-tuning
Model : roberta-base (125 M parameters)
Tasks : SST-2, CoLA, SNLI, SQuAD 2.0
Total runs : 32 (4 tasks x 8 conditions)
GPU needed : T4 (Colab free tier) or better

Generated by generate_colab_guide.py

Overview

This guide walks you through running the full roberta-base LoRA rank sweep experiment on Google Colab. The experiment trains roberta-base on four NLP tasks across eight LoRA rank conditions ($r = 1, 2, 4, 8, 16, 32, 64$, plus a full fine-tuning baseline), producing 32 training runs in total.

Each run trains for 10 full epochs with no early stopping, logging the test metric every 200 gradient steps. This fixed-length design is intentional: it lets you observe whether performance stays stable past its peak (the expected behaviour at the correct rank R^*) or degrades (overfitting, which signals the rank is higher than necessary).

What You Will Need

- A Google account with access to Google Colab (free tier is sufficient for most tasks)
- Approximately 5 GB of free Google Drive space for results, checkpoints, and plots
- Your experiment repository pushed to a public (or accessible) GitHub repo
- No local GPU or local Python environment is required

Estimated Total Time (Colab Free T4 GPU)

The table below shows approximate training time per task. Times are estimates for a T4 GPU running roberta-base at the given batch sizes.

Task	Train examples	Steps / epoch	Total steps (10 ep)	Per rank (T4)	8 ranks (T4)
SST-2	~67 k	~2,094	~20,940	~8 min	~65 min
CoLA	~8.5 k	~266	~2,660	~1 min	~10 min
SNLI	~549 k	~17,156	~171,560	~55 min	~7-8 hrs
SQuAD2	~130 k	~10,000+	~100k+	~60 min	~8+ hrs

Note: SNLI and SQuAD 2.0 each take many hours across all 8 rank conditions. A single free Colab session allows up to ~12 hours of GPU time (less on the free tier). Run one task at a time using the `--task` flag and save results to Drive after each completed task.

Note: Colab Pro / Pro+ provide longer sessions and more GPU memory, which makes running SNLI and SQuAD 2.0 much more practical.

Resume support: The experiment runner includes built-in resume logic: if a session disconnects mid-run, re-running the same command skips any already-completed rank conditions (tracked via `run_summary.json`). Always save results to Drive before a session ends.

Step-by-Step Instructions

STEP 1 Open Google Colab and Set the Runtime to GPU

Go to <https://colab.research.google.com> and create a new notebook.

Then set the GPU runtime:

- Click Runtime in the menu bar
- Select Change runtime type
- Set Hardware accelerator to T4 GPU (or A100 if available)
- Click Save

Verify the GPU is active by running the following cell:

Cell 1 — Verify GPU

```
!nvidia-smi
import torch
print("CUDA available:", torch.cuda.is_available())
print("Device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")
```

Note: If nvidia-smi shows no GPU, the runtime type was not set correctly. Repeat the steps above.

STEP 2 Mount Google Drive

Mounting Drive is strongly recommended. Colab sessions disconnect without warning, and any results stored only in /content/ are permanently lost. Drive persists data across sessions.

Cell 2 — Mount Drive

```
from google.colab import drive
drive.mount('/content/drive')

# Create a folder in Drive to hold all experiment data
import os
DRIVE_DIR = '/content/drive/MyDrive/lora_experiment'
os.makedirs(DRIVE_DIR, exist_ok=True)
print("Drive folder ready:", DRIVE_DIR)
```

STEP 3 Clone the Repository

Clone your GitHub repository into the Colab runtime's local storage. Replace YOUR_USERNAME and YOUR_REPO_NAME with your actual GitHub details.

Cell 3 — Clone repository

```
# Replace these with your actual GitHub username and repo name
GITHUB_USER = "YOUR_USERNAME"
REPO_NAME = "YOUR_REPO_NAME"

!git clone https://github.com/{GITHUB_USER}/{REPO_NAME}.git
%cd {REPO_NAME}
```

```
!ls -la    # confirm the files are present
```

Note: If your repository is private, you will need to authenticate. Use a GitHub personal access token (PAT): `git clone https://YOUR_TOKEN@github.com/USER/REPO.git`

Working directory: After cloning, Colab's working directory is set to the repo root. All subsequent commands should be run from there.

STEP 4 Install Dependencies

Colab comes with PyTorch, NumPy, and several ML libraries pre-installed. The remaining packages can be installed in one cell. This typically takes 2-4 minutes.

Cell 4 — Install packages

```
!pip install -q \
    transformers>=4.44.0 \
    peft>=0.12.0 \
    datasets>=2.21.0 \
    evaluate>=0.4.3 \
    scikit-learn>=1.5.0 \
    seaborn>=0.13.0 \
    scipy>=1.14.0 \
    fpdf2>=2.7.0

# torch, numpy, pandas, matplotlib, tqdm are already in Colab
print("All packages installed.")
```

Note: bitsandbytes and accelerate are only needed for the Llama models (which are not part of the roberta-base runs). Do not install them unless you are extending to Phase 2 multi-model experiments.

STEP 5 Restore Previous Results (if resuming)

If you previously ran part of the experiment and saved results to Drive, copy them back to the runtime before starting. This allows the resume logic to pick up where the last session left off.

Cell 5 — Restore results from Drive (skip if starting fresh)

```
import shutil, os

DRIVE_RESULTS = '/content/drive/MyDrive/lora_experiment/results'
LOCAL_RESULTS = 'results'

if os.path.exists(DRIVE_RESULTS):
    shutil.copytree(DRIVE_RESULTS, LOCAL_RESULTS, dirs_exist_ok=True)
    print("Results restored from Drive.")
else:
    print("No previous results found in Drive. Starting fresh.")
```

STEP 6 Run the Experiment

Run the experiment one task at a time. This is the safest approach: save results to Drive after each task and start the next task in the same or a fresh session. The `--task` flag restricts the run to a single task.

Option A — Recommended: One Task at a Time

Start with CoLA (fastest, ~10 minutes total) to verify everything works:

Cell 6a — Run CoLA (fastest, ~10 min)

```
!python run_experiment.py --model roberta-base --task cola 2>&1 | tee cola_log.txt
```

Then run SST-2 (~65 minutes total):

Cell 6b — Run SST-2 (~65 min)

```
!python run_experiment.py --model roberta-base --task sst2 2>&1 | tee sst2_log.txt
```

Then SNLI (~7-8 hours total). Consider using a Colab Pro session or overnight run:

Cell 6c — Run SNLI (~7-8 hrs)

```
!python run_experiment.py --model roberta-base --task snli 2>&1 | tee snli_log.txt
```

Finally SQuAD 2.0 (~8+ hours total):

Cell 6d — Run SQuAD 2.0 (~8+ hrs)

```
!python run_experiment.py --model roberta-base --task squad2 2>&1 | tee squad2_log.txt
```

Option B — All Tasks at Once (risky on free tier)

If you have a long GPU session (Colab Pro/Pro+ with an A100), you can run all 32 conditions in a single command. The experiment runner will log each completed run to `run_summary.json` so nothing is double-counted if the session disconnects.

Cell 6e — All tasks at once (Pro tier only)

```
!python run_experiment.py --model roberta-base 2>&1 | tee full_experiment_log.txt
```

Note: Even with a Pro session, save results to Drive frequently. Use the cell in Step 7 after every task finishes.

STEP 7 Save Results to Google Drive

Run this cell after each completed task (or after any significant progress). It copies the results directory from the Colab runtime to your Drive folder.

Cell 7 — Save results to Drive

```
import shutil

DRIVE_DIR = '/content/drive/MyDrive/lora_experiment'
shutil.copytree('results', f'{DRIVE_DIR}/results', dirs_exist_ok=True)
print("Results saved to Drive.")

# Optionally save the training logs too
import os
for log in ['cola_log.txt', 'sst2_log.txt', 'snli_log.txt', 'squad2_log.txt']:
    if os.path.exists(log):
        shutil.copy(log, f'{DRIVE_DIR}/{log}')
        print(f" Saved {log}")
```

Note: Do not wait until the end of a long session to save. If Colab disconnects, any unsaved local data is lost. Each completed run writes its CSV to results/ immediately, so saving to Drive preserves the work done so far.

STEP 8 Run Analysis and Generate Plots

Once all (or some) tasks are complete, run the analysis script to compute requisite rank R^* and generate all plots. You can run analysis at any point; it will work with however many results are available.

Cell 8 — Run analysis

```
!python analysis.py --model roberta-base

# Plots are saved to plots/roberta-base/
!ls plots/roberta-base/
```

Plot output: The analysis script produces three plot types: (1) rank sweep curves per task showing metric vs. LoRA rank, (2) a bar chart of R^* per task, and (3) a cross-model heatmap (only when multiple model results exist).

STEP 9 Download Results and Plots

When you are finished, zip everything up and download it to your local machine, or simply access it from Google Drive.

Cell 9 — Zip and download

```
# Save plots to Drive
import shutil
DRIVE_DIR = '/content/drive/MyDrive/lora_experiment'
shutil.copytree('plots', f'{DRIVE_DIR}/plots', dirs_exist_ok=True)
print("Plots saved to Drive.")

# Or download a zip directly to your browser
!zip -r experiment_output.zip results/ plots/
from google.colab import files
files.download('experiment_output.zip')
```

Handling Disconnections and Resuming

Google Colab sessions disconnect automatically after periods of inactivity or when they exceed the session time limit. The experiment runner is designed to handle this gracefully.

How the Resume Logic Works

After every completed run (one task + one rank condition), the runner writes an entry to:

Resume tracking file

```
results/roberta-base/run_summary.json
```

Each entry records the task name, rank condition, final metric, and status (done or error). When you re-run the experiment command, the runner reads this file and skips any run already marked as done.

Resuming After a Disconnection

In a new Colab session, follow these steps:

- Step 1: Mount Drive (Cell 2 again)
- Step 2: Re-clone the repo and install packages (Cells 3-4 again)
- Step 3: Restore previous results from Drive (Cell 5)
- Step 4: Re-run the same experiment command — completed runs are skipped automatically

Resume example — continue SST-2 after a disconnection

```
# In a new session:
from google.colab import drive
drive.mount('/content/drive')

import shutil, os
DRIVE_RESULTS = '/content/drive/MyDrive/lora_experiment/results'
shutil.copytree(DRIVE_RESULTS, 'results', dirs_exist_ok=True)
print("Restored", len(os.listdir('results/roberta-base')), "task folders")

# Re-run — already-done ranks are skipped
!python run_experiment.py --model roberta-base --task sst2
```

Note: If you want to re-run a specific rank condition that previously errored, open `results/roberta-base/run_summary.json`, find the entry, and delete it or change its status from 'error' to anything other than 'done'. Then re-run the experiment command.

Checking Progress Mid-Run

You can inspect the summary file at any time to see which runs have completed:

Check progress

```
import json, os

summary_path = 'results/roberta-base/run_summary.json'
if os.path.exists(summary_path):
```

```
with open(summary_path) as f:
    summary = json.load(f)
done = [(k, v) for k, v in summary.items() if v['status'] == 'done']
error = [(k, v) for k, v in summary.items() if v['status'] == 'error']
print(f"Completed: {len(done)}/32")
print(f"Errors : {len(error)}")
for k, v in done:
    print(f" {k:20s} metric={v['final_metric']} elapsed={v['elapsed_s']:.0f}s")
else:
    print("No summary file yet – no runs completed.")
```

Troubleshooting

Q: ModuleNotFoundError: No module named 'peft' (or similar)

A: The packages were not installed, or Colab restarted the kernel after installation. Re-run Cell 4 (pip install) and then re-run the experiment cell. You do NOT need to re-clone the repo.

Q: CUDA out of memory

A: This should not happen with roberta-base on a T4 (the model is only 125 M parameters). If it does occur, restart the runtime (Runtime > Restart runtime) and re-run from Cell 5 (restore results). If it persists, reduce BATCH_SIZE_CLS in src/train.py from 32 to 16.

Q: Session disconnected before results were saved to Drive

A: If you did not run Cell 7, the results from that session are lost and those rank conditions must be re-run. The completed ones (that were saved in a previous session) are safe. This is why saving to Drive after each task is strongly recommended.

Q: HuggingFace dataset download is slow or times out

A: The first run for each task downloads the dataset from HuggingFace Hub. HuggingFace caches datasets in ~/.cache/huggingface/. If the session restarts, the cache is lost and the download repeats. This is unavoidable on the free tier — the dataset cache cannot easily be stored on Drive.

Q: git clone fails (authentication error for a private repo)

A: Use a GitHub personal access token in the clone URL: `https://YOUR_TOKEN@github.com/USER/REPO.git`. Generate a PAT at `github.com > Settings > Developer settings > Personal access tokens`.

Q: run_summary.json shows status='error' for some runs

A: Look at the printed traceback from the training log (cola_log.txt etc.) to diagnose the cause. Once fixed, delete the 'error' entry from run_summary.json and re-run the experiment command.

Q: analysis.py raises FileNotFoundError or produces empty plots

A: Analysis requires at least one completed training_log.csv in results/. Check that the experiment ran successfully before running analysis. You can run analysis on partial results (e.g., just CoLA + SST-2).

Quick Reference

Useful Commands

Run all tasks

```
python run_experiment.py --model roberta-base
```

Run one task only

```
python run_experiment.py --model roberta-base --task cola
```

Run one specific rank

```
python run_experiment.py --model roberta-base --rank 8
```

Single run (debug)

```
python run_experiment.py --model roberta-base --task sst2 --rank 4
```

Run analysis

```
python analysis.py --model roberta-base
```

Check GPU

```
nvidia-smi
```

View training log (CoLA, r=8)

```
cat results/roberta-base/cola/8/training_log.csv
```

View run summary

```
cat results/roberta-base/run_summary.json
```

Results Directory Structure

After a complete run, the results directory looks like this:

```
results/
  roberta-base/
    run_summary.json          <- status of every run (32 entries when complete)
    sst2/
      1/  training_log.csv    <- step, train_loss, test_metric, exact_match
          checkpoint/        <- saved model weights + tokenizer
      2/  training_log.csv
      ...
      64/ training_log.csv
      full/training_log.csv
    cola/
      1/ ... full/           <- same structure
    snli/
      1/ ... full/
    squad2/
      1/ ... full/           <- test_metric = F1, exact_match = EM

  plots/
    roberta-base/
```

```
sst2_rank_sweep.png  
cola_rank_sweep.png  
snli_rank_sweep.png  
squad2_rank_sweep.png
```

Rank Conditions

There are 8 rank conditions per task: LoRA ranks $r = 1, 2, 4, 8, 16, 32, 64$ (7 conditions) plus one full fine-tuning baseline, giving $8 * 4 = 32$ total runs.

- LoRA runs: only the adapter parameters are trained (~0.1-2% of total)
- Full run : all 125 M parameters are trained (100%)
- `lora_alpha` is always $2 * \text{rank}$ — keeps the effective output scale constant

Key Hyperparameters

Learning rate:	<code>2e-5 (AdamW)</code>
Weight decay:	<code>0.01</code>
Epochs:	<code>10 (no early stopping)</code>
Eval frequency:	<code>every 200 gradient steps</code>
Batch size (cls):	<code>32</code>
Batch size (QA):	<code>16 (SQuAD sequences are 384 tokens)</code>
Warmup:	<code>6% of total steps</code>
Grad clip:	<code>max norm 1.0</code>
LoRA dropout:	<code>0.1</code>
LoRA alpha:	<code>rank * 2 (constant effective scale = 2)</code>
LoRA targets:	<code>query, key, value, dense (all 4 attention projections)</code>